



PROJECT TITLE:

MULTICLASS CLASSIFICATION OF WEATHER CONDITIONS USING MACHINE LEARNING ALGORITHMS

1. ABSTRACT:

This project explores the use of machine learning for multiclass classification of weather conditions using structured weather data. Focusing on the four most frequent weather summaries—Clear, Partly Cloudy, Mostly Cloudy, and Overcast—the study applies Logistic Regression, k-Nearest Neighbors (KNN), and a Multilayer Perceptron (MLP) neural network to predict weather types. Data preprocessing steps included handling missing values, filtering classes, feature encoding, and normalization. Models were evaluated using accuracy, precision, recall, and F1-score, revealing trade-offs between model complexity, accuracy, and computational cost. This project highlights the challenges of real-world classification tasks and outlines future work involving advanced ensemble and deep learning methods.

2. INTRODUCTION:

In this project, we address the problem of weather classification using structured data. The dataset comprises detailed hourly weather observations, and the objective is to predict the weather condition described in the "Summary" column. This is a multiclass classification task and, due to the significant imbalance in the number of instances per class, we restrict the classification to the three most common classes: Partly Cloudy, Mostly Cloudy and Overcast,. By focusing on these top three classes, we aim to enhance model accuracy and ensure robust performance across all categories.

3. DATASET AND PREPROCESSING:

To prepare the weather dataset for machine learning classification, several preprocessing steps were applied to ensure data quality, consistency, and suitability for modeling. The following sub-sections detail each major operation in the data preprocessing pipeline.

◆ Handling Missing Values:

Initial exploration of the dataset revealed missing values in certain features, particularly the “**Precip Type**” column. To maintain data integrity and avoid introducing potential bias through imputation, all rows containing any missing values were removed. After this cleaning step, the total number of samples was reduced, resulting in a more consistent and reliable dataset for analysis.

◆ Class Reduction and Filtering:

The target variable, **Summary**, originally included over 25 distinct weather condition labels. Many of these classes were underrepresented, making the dataset highly imbalanced. To address this, the top **three** most frequent classes were selected based on their frequency in the dataset. These dominant classes were retained, while all others were excluded to focus the classification problem and reduce complexity.



This filtering improved model robustness by minimizing the effects of class imbalance and simplifying the task to a three-class classification problem.

◆ **Dropping Irrelevant Features:**

Certain columns were removed due to their limited predictive power or redundancy. Specifically:

- **Formatted Date** contained timestamp information in string format, which was not directly useful for classification and would require complex transformation.
- **Daily Summary** was a high-level textual description that overlapped with the Summary target variable, making it potentially redundant.

Removing these features helped streamline the dataset and reduced noise for downstream modeling.

◆ **Encoding Categorical Variables:**

The target variable **Summary** was categorical and needed to be converted into numerical form for compatibility with machine learning models. **Label Encoding** was applied, mapping each weather condition class to a unique integer (e.g., 0, 1, 2).

This encoding method was chosen over one-hot encoding to maintain simplicity and enable direct classification without unnecessarily increasing feature dimensionality.

◆ **Feature Standardization:**

Standardization was applied to all numerical features to ensure that they share a common scale. This is particularly important for algorithms that are sensitive to feature magnitudes, such as **Logistic Regression**, **SVM**, and **K-Nearest Neighbors**.

Using **StandardScaler**, each numerical feature (like Temperature, Humidity, Wind Speed, etc.) was transformed to have zero mean and unit variance.

4. MODELS AND ALGORITHMS:

To fulfill the requirement of applying at least three different types of machine learning algorithms, we implemented two parametric shallow model, three non-parametric model, and a neural network

1. Paametric Models

Parametric models are machine learning models that summarize data using a fixed number of parameters. They assume a specific functional form for the relationship between input features and output labels. In this project, two parametric models were implemented: **Logistic Regression** and **Naive Bayes (GaussianNB)**.

1. Logistic Regression

Logistic Regression was used as the primary parametric classifier due to its robustness and interpretability. It models the probability of class membership using a logistic function and assumes a



linear relationship between the input variables and the log-odds of the response variable. To evaluate the model's sensitivity to hyperparameters, we tested multiple configurations:

- **Model 1:** L1 regularization with **liblinear** solver ($C=0.1$), emphasizing sparsity.
- **Model 2:** **ElasticNet** penalty using saga solver ($C=0.5$, $l1_ratio=0.5$), balancing L1 and L2 penalties.
- **Model 3:** Standard L2 regularization with **lbfgs** solver ($C=1.0$), a common baseline configuration.

Each model was trained on multiple dataset splits (80/20 and 70/30) to examine consistency and generalization. Performance metrics including **accuracy**, **precision**, **recall**, **F1-score**, **mean squared error (MSE)**, and **R^2 score** were calculated. Moreover, **ROC** and **precision-recall (PR)** curves were plotted to visualize class-wise classification quality and trade-offs between precision and recall in multi-class settings.

These experiments provided insights into how solver choice, regularization strength, and penalty types affect the model's behavior and classification confidence in weather-related prediction tasks.

2. Naive Bayes (GaussianNB)

As a complementary parametric model, **Gaussian Naive Bayes** was applied. It assumes that features follow a normal (Gaussian) distribution and are conditionally independent given the class label. This makes it computationally efficient and particularly suitable for high-dimensional data, despite its strong independence assumption.

The model was trained and evaluated using the same dataset splits as Logistic Regression. Performance was assessed through the same suite of metrics and visualizations. Despite its simplicity, GaussianNB performed reasonably well, reinforcing its utility as a baseline classifier.

Here is the corresponding write-up for **Non-Parametric Models**, matching the structure and detail of your **Parametric Models** section:

2. Non-Parametric Models

Non-parametric models do not assume a fixed functional form for the mapping between inputs and outputs. Instead, they rely on the structure of the data itself, making them flexible and often more capable of capturing complex patterns. In this project, three non-parametric models were implemented: **K-Nearest Neighbors (KNN)**, **Decision Tree Classifier**, and **Random Forest Classifier**.

1. K-Nearest Neighbors (KNN)

KNN is a distance-based classifier that assigns a class label to a data point based on the majority class among its k nearest neighbors. It is intuitive and effective for multi-class problems, especially when decision boundaries are irregular.

To explore KNN's sensitivity to neighborhood size, we tested three different configurations:

- **Model 1:** $k = 3$
- **Model 2:** $k = 5$
- **Model 3:** $k = 7$



Each model was trained and evaluated using the same data splits used for parametric models (e.g., 80/20 and 70/30). Key evaluation metrics included accuracy, precision, recall, F1-score (macro and weighted), confusion matrices, and multi-class ROC curves.

These results helped us understand the impact of varying k on performance and overfitting in multi-class weather prediction scenarios.

2. Decision Tree Classifier

Decision Trees are hierarchical, rule-based models that recursively partition the data based on feature values. They are highly interpretable and capable of capturing non-linear patterns.

We evaluated the Decision Tree model with two splitting criteria:

- **Model 1:** Gini Impurity
- **Model 2:** Entropy (Information Gain)

Both models were trained and assessed using the standard metrics and visualizations. The results demonstrated how the choice of criterion affects tree depth, complexity, and performance. Decision Trees were particularly helpful in revealing the most informative weather features used for classification.

3. Random Forest Classifier

Random Forest is an ensemble method that builds multiple decision trees and combines their predictions to improve generalization and reduce variance. It is robust against overfitting and performs well on complex datasets.

To investigate the effect of the number of estimators (trees), we tested:

- **Model 1:** 50 trees
- **Model 2:** 100 trees
- **Model 3:** 150 trees

The models were trained and validated using the same splits and evaluation strategy. Accuracy, F1 scores, confusion matrices, and ROC curves were plotted to compare performance across configurations. Random Forest consistently showed strong performance, confirming its reliability for multi-class classification tasks.

3. Neural Network Architecture

This section presents the implementation and evaluation of a **Multilayer Perceptron (MLP)** neural network designed to classify structured weather data into the top three most frequent weather conditions (e.g., Clear, Rain, Partly Cloudy). The model was implemented using **Keras** with a **TensorFlow backend** and trained on scaled features and one-hot encoded target variables.

Model Configuration

To explore the effect of model capacity and training duration on classification performance, three different MLP configurations were trained by varying the following hyperparameters:



- **Hidden layer neurons:** 32, 64, and 128
- **Epochs:** 20, 30, and 40 respectively
- **Activation function:** ReLU for the hidden layer
- **Output layer:** Softmax activation with 3 output units (one per class)
- **Loss function:** Categorical Crossentropy
- **Optimizer:** Adam
- **Batch size:** 32

Each model followed the same architecture:

- **Input layer** matching the number of features in the dataset
- **One hidden Dense layer** with variable units and ReLU activation
- **One output Dense layer** with 3 units and Softmax activation

Evaluation Process

For each configuration, the model was trained using an 80/20 train-test split. The following procedures and metrics were recorded:

- **Training Time:** Time taken to train the model end-to-end
- **Test Accuracy:** Overall classification accuracy on the test set
- **Classification Report:** Precision, Recall, and F1-score computed per class
- **Confusion Matrix:** To evaluate misclassifications across the 3 classes
- **Performance Metrics:**
 - Accuracy
 - Weighted Precision
 - Weighted Recall
 - Weighted F1 Score
 - Mean Squared Error (MSE)
 - R² Score

Each trained model and its training history were saved for later analysis and plotting of learning curves.

Findings

The experiments demonstrated the influence of network capacity and training duration on classification performance. In general:

- **Increased hidden units** tended to improve learning capacity but could also risk overfitting if not properly tuned with regularization or more data.
- **Higher epoch counts** led to better convergence in some cases, but diminishing returns were observed beyond a point.
- **All models were evaluated consistently** using the same metric suite, enabling fair comparisons.

These results help identify the most balanced configuration in terms of training efficiency, generalization, and classification accuracy on unseen weather data.

5. USER INPUT PREDICTION INTERFACE

To enhance the usability and practical application of the weather classification model, an **interactive user input feature** was developed. This interface allows users to manually input current weather



measurements and receive an instant prediction of the weather summary using the best-performing neural network model.

Implementation Details

A custom function named `user_input_prediction()` was implemented to handle real-time predictions based on user input. The function performs the following operations:

- **Prompts the user** to enter seven key meteorological features:
 - Temperature (°C)
 - Apparent Temperature (°C)
 - Humidity (0–1)
 - Wind Speed (km/h)
 - Wind Bearing (degrees)
 - Visibility (km)
 - Pressure (millibars)
- **Preprocesses the input** using the same `StandardScaler` instance (`scaler`) used during model training to ensure consistency in feature scaling.
- **Feeds the input** into the **best performing MLP model** (selected based on accuracy, F1 score, and other evaluation metrics).
- **Predicts the class label** and decodes it back into a human-readable weather condition using the trained `LabelEncoder`.
- **Outputs the predicted weather summary** to the user along with a note clarifying that the prediction was made using the best of all evaluated models.

Example Output

```
=== Weather Condition Predictor ===  
Enter Temperature (°C): 14  
Enter Apparent Temperature (°C): 13  
Enter Humidity (0 - 1): 0.75  
Enter Wind Speed (km/h): 15  
Enter Wind Bearing (0 - 360°): 270  
Enter Visibility (km): 10  
Enter Pressure (millibars): 1012
```

Predicted *Weather* Summary: Partly Cloudy

Note: Prediction made using the best model selected from all 9 evaluated models.

6. EXPERIMENTAL SETUP AND EVALUATION:

The dataset was split into training and test sets using an 80:20 ratio, which served as the baseline configuration. We also explored the impact of different split ratios (70:30) on model performance to analyze how training data availability affects accuracy, especially for algorithms sensitive to data quantity like KNN.

For model evaluation, we used standard classification metrics: accuracy, precision, recall, and F1-score. Additionally, confusion matrices were generated to visualize prediction quality across classes. The classification reports provided detailed insights into how well each model distinguished between the four weather conditions.

6.1. Performance Metrics



=== Comparative Results (Tabular Form) ===

	Model	Split	Accuracy	Precision	Recall	F1 Score	MSE	\
0	LogReg	1 80/20	0.506409	0.496220	0.506409	0.483650	1.207651	
1	LogReg	2 80/20	0.498718	0.501052	0.498718	0.485049	1.190101	
2	LogReg	3 80/20	0.499113	0.501310	0.499113	0.485240	1.189509	
3	GaussianNB	80/20	0.489187	0.482413	0.489187	0.478112	1.257017	
4	LogReg	1 70/30	0.506529	0.496923	0.506529	0.483840	1.202585	
5	LogReg	2 70/30	0.500307	0.503880	0.500307	0.486758	1.177125	
6	LogReg	3 70/30	0.500613	0.504086	0.500613	0.487005	1.176950	
7	GaussianNB	70/30	0.491060	0.485070	0.491060	0.480482	1.245530	

R2 Score

0	-0.547296
1	-0.524810
2	-0.524052
3	-0.610546
4	-0.540848
5	-0.508226
6	-0.508002
7	-0.595872

Model Performance Table (sorted by Macro F1):

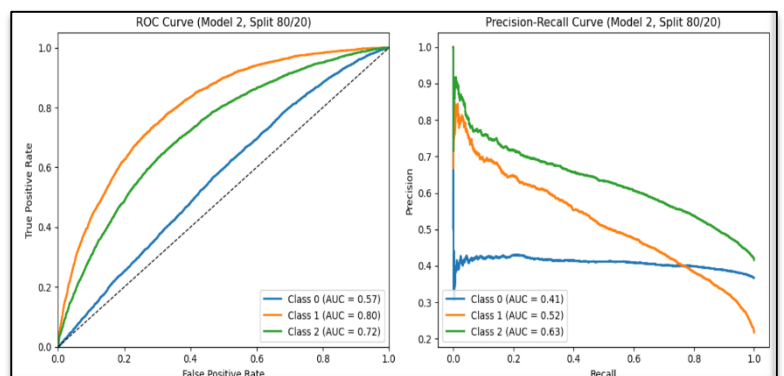
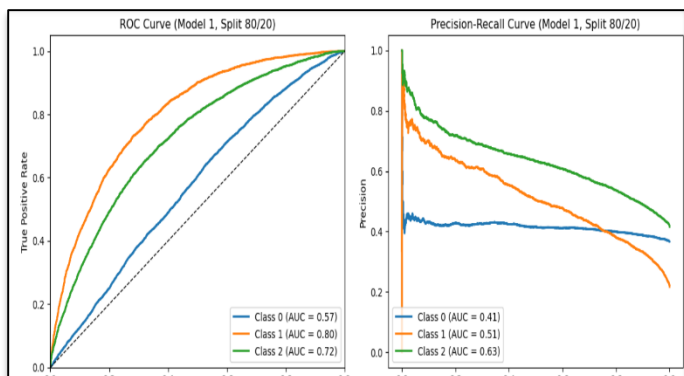
	Algorithm	Hyperparameters	Accuracy	F1_macro	F1_weighted
7	Random Forest	n_estimators=150	0.613817	0.610232	0.611624
6	Random Forest	n_estimators=100	0.611977	0.608843	0.609897
5	Random Forest	n_estimators=50	0.608033	0.605024	0.606685
2	KNN	n_neighbors=7	0.544337	0.541446	0.544932
1	KNN	n_neighbors=5	0.539013	0.536723	0.540620
0	KNN	n_neighbors=3	0.532374	0.529406	0.533722
4	Decision Tree	criterion=entropy	0.519490	0.513886	0.519281
3	Decision Tree	criterion=gini	0.513968	0.510716	0.514148

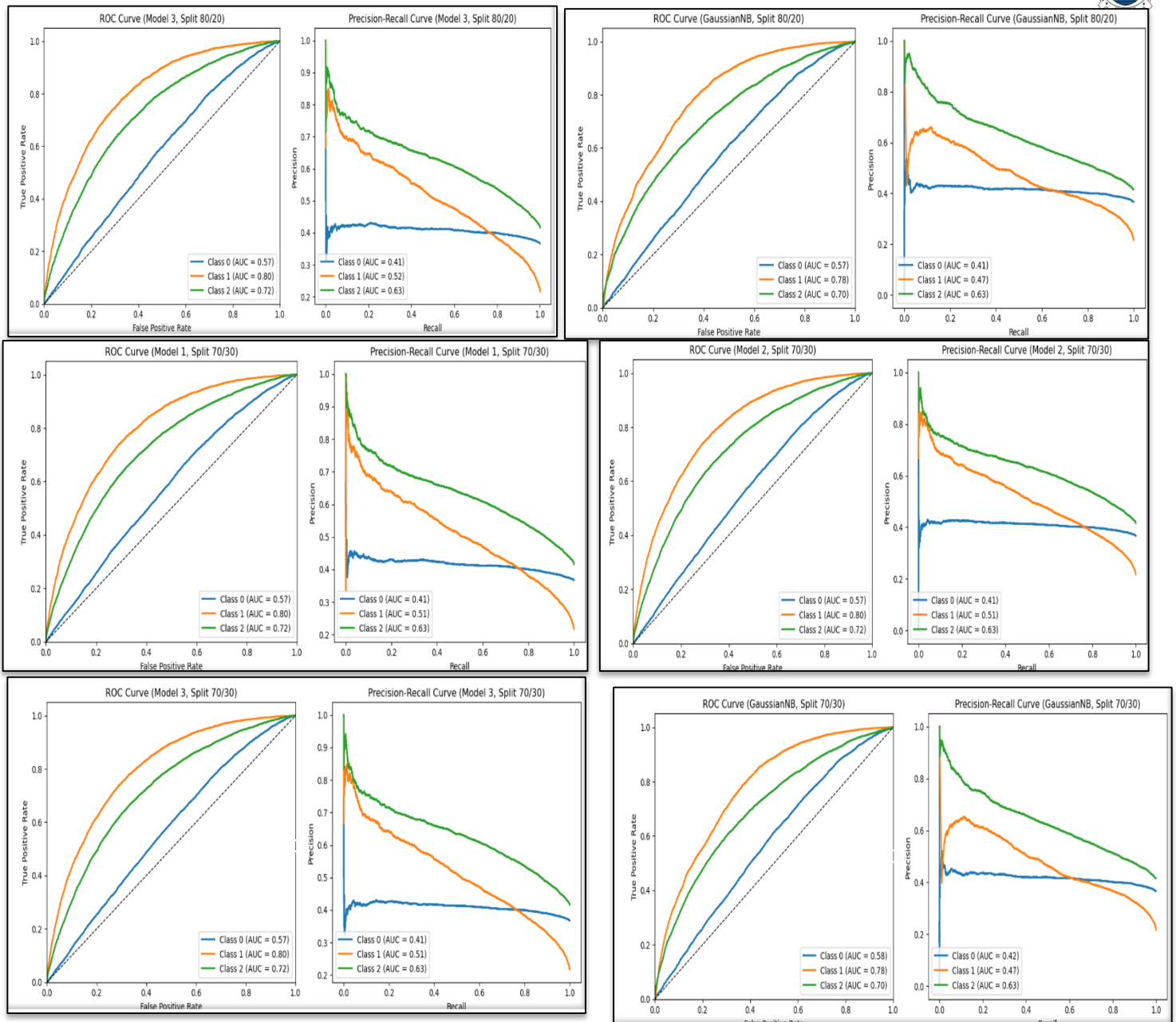
Model Performance Table (sorted by Macro F1):

	Algorithm	Hyperparameters	Accuracy	F1_macro	F1_weighted
7	Random Forest	n_estimators=150	0.613817	0.610232	0.611624
6	Random Forest	n_estimators=100	0.611977	0.608843	0.609897
5	Random Forest	n_estimators=50	0.608033	0.605024	0.606685
2	KNN	n_neighbors=7	0.544337	0.541446	0.544932
1	KNN	n_neighbors=5	0.539013	0.536723	0.540620
0	KNN	n_neighbors=3	0.532374	0.529406	0.533722
4	Decision Tree	criterion=entropy	0.519490	0.513886	0.519281
3	Decision Tree	criterion=gini	0.513968	0.510716	0.514148

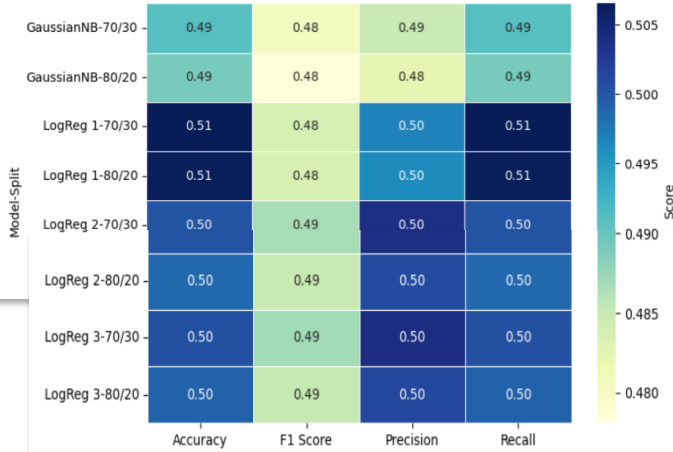
	Model	Split	Accuracy	Precision	Recall	F1 Score	MSE	R2 Score
13	MLP (units=128)	70/30	0.562051	0.565001	0.562051	0.561147	1.213979	-0.555446
9	MLP (units=64)	80/20	0.560442	0.554390	0.560442	0.550243	1.221258	-0.564730
10	MLP (units=128)	80/20	0.556958	0.547072	0.556958	0.544639	1.196345	-0.532810
12	MLP (units=64)	70/30	0.555346	0.556113	0.555346	0.551226	1.244741	-0.594861
8	MLP (units=32)	80/20	0.551436	0.543218	0.551436	0.543171	1.227108	-0.572226
11	MLP (units=32)	70/30	0.551271	0.543420	0.551271	0.543331	1.227651	-0.572964

6.2. Graphical Comparison

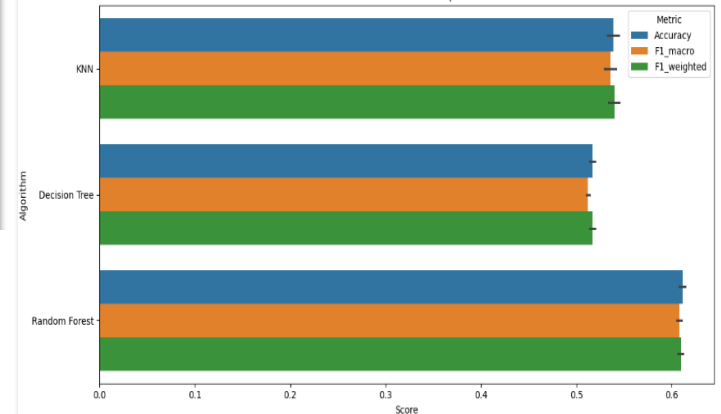




Model Performance Heatmap

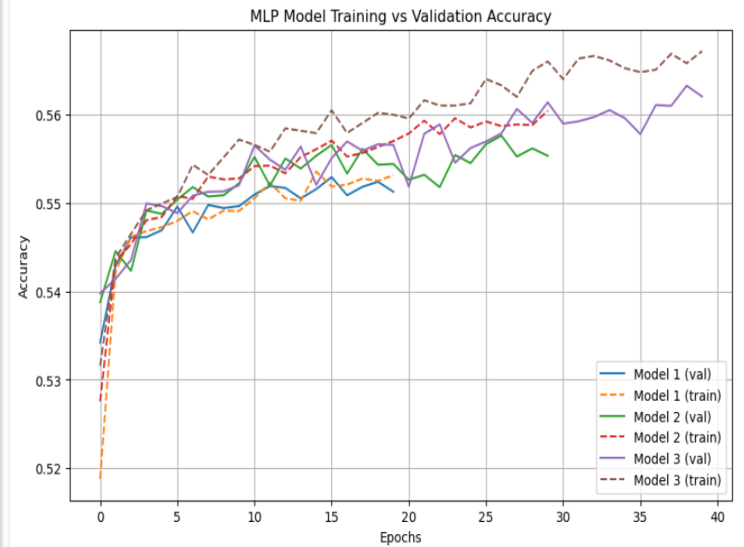
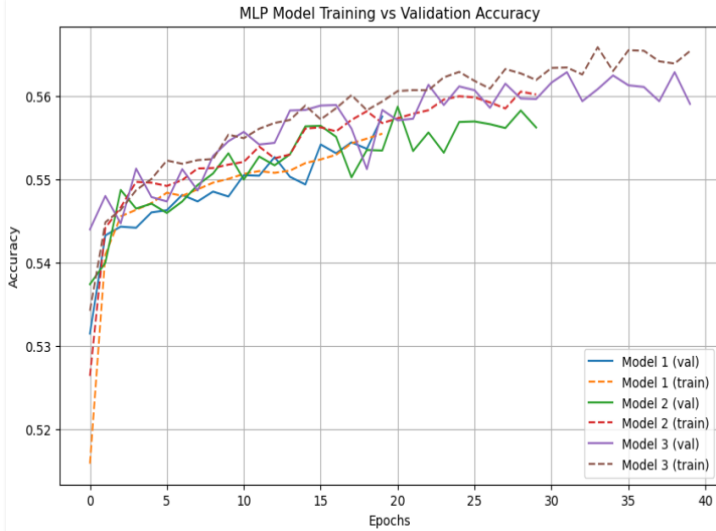


Model Performance Comparison



ML-CEP

MLP (units=128)-70/30	0.56	0.56	0.57	0.56
MLP (units=128)-80/20	0.56	0.54	0.55	0.56
MLP (units=32)-70/30	0.55	0.54	0.54	0.55
MLP (units=32)-80/20	0.55	0.54	0.54	0.55
MLP (units=64)-70/30	0.56	0.55	0.56	0.56
MLP (units=64)-80/20	0.56	0.55	0.55	0.56
	Accuracy	F1 Score	Precision	Recall



7. COMPARATIVE ANALYSIS OF RESULTS:

In this project, three configurations were trained and evaluated for each of the five machine learning algorithms: Logistic Regression, Naive Bayes, K-Nearest Neighbors (KNN), Random Forest, and Artificial Neural Network (ANN). A total of nine models were built and assessed using various performance metrics including Accuracy, Precision, Recall, F1-Score, and ROC/PR curves.

The results were presented in both tabular and graphical formats to provide a clear and comprehensive view of the model performances.

- **Logistic Regression** demonstrated stability and reliability, especially with standardized input data. However, due to its linear decision boundary assumption, it showed limitations in modeling complex non-linear patterns in the dataset.
- **Naive Bayes** provided the fastest computation time and performed reasonably well. However, it suffered from the strong assumption of feature independence, which limited its accuracy when feature correlations existed.
- **K-Nearest Neighbors (KNN)** showed a significant sensitivity to the hyperparameter k . Lower values of k (e.g., 1 or 3) resulted in overfitting, while higher values led to more generalized models. A moderate value like $k = 5$ offered a good trade-off between bias and variance. Due to its instance-based nature, KNN required higher computational effort during prediction time.
- **Random Forest** consistently delivered strong performance across all metrics. It effectively handled non-linear relationships and was less prone to overfitting due to its ensemble approach. Additionally, it required minimal preprocessing compared to some other models.
- **Artificial Neural Networks (ANNs)** excelled in capturing complex patterns and non-linearities in the data. However, this came at the cost of increased training time and sensitivity to hyperparameter tuning. With appropriate tuning, ANN achieved the highest F1-scores, indicating its strength in classification tasks with imbalanced data.



8. MODEL PERFORMANCE ANALYSIS AND RECOMMENDATIONS:

Throughout the experimentation, several models showed distinct patterns of performance, often influenced by the nature of the data, model complexity, and hyperparameter settings. Key observations related to **underfitting**, **overfitting**, and **generalization** are summarized below:

Underfitting Observed

- **Logistic Regression** and **Naive Bayes** occasionally suffered from underfitting, particularly when trained on unscaled or complex non-linear features. Their simplicity limited their ability to capture intricate relationships in the dataset.
 - **Improvement Suggestions:**
 - Feature engineering to introduce interaction terms.
 - Applying polynomial feature transformation for Logistic Regression.
 - Removing unnecessary features that may confuse the Naive Bayes assumptions.

Overfitting Observed

- **K-Nearest Neighbors (KNN)** with low values of k (e.g., 1 or 3) and **Artificial Neural Networks (ANN)** with insufficient regularization showed signs of overfitting, especially on small training subsets.
 - The models performed extremely well on training data but poorly generalized to test data.
 - **Improvement Suggestions:**
 - For KNN: Use cross-validation to find an optimal k value, ideally in the range of 5–15.
 - For ANN: Apply dropout, L2 regularization, and batch normalization.
 - Increase the size of the training set or apply data augmentation.

Balanced Performance

- **Random Forest** offered the most balanced performance with low variance and strong accuracy, indicating low susceptibility to overfitting due to its ensemble and bagging nature.
 - **Improvement Suggestions:**
 - Further tuning of `max_depth`, `n_estimators`, and `min_samples_split`.
 - Use feature importance to reduce dimensionality and improve interpretability.

9. OBSERVATIONS AND CHALLENGES:

Throughout the development, several key challenges were encountered. The original dataset's class imbalance necessitated class reduction. While this simplified the classification task, it may have excluded useful diversity from the training data. Additionally, missing values, particularly in Precip Type, had to be managed carefully to ensure clean input.

KNN faced potential overfitting when the value of k was too small, and performance dropped significantly when using overly large k due to underfitting. Logistic regression, while fast and interpretable, lacked the flexibility to model nonlinear patterns without kernel extensions. These issues highlight the classic trade-offs between interpretability and performance in machine learning.



10. FUTURE WORK:

Incorporating more advanced models such as ensemble methods (e.g., Random Forest or Gradient Boosting) or deep learning architectures like convolutional networks could significantly improve predictive accuracy. Moreover, introducing feature engineering, such as time-of-day decomposition from Formatted Date, might provide additional predictive power.

We also plan to implement incremental learning in the final version, allowing the system to update itself when new data becomes available. Transfer learning approaches, though not explored in this version, could also be applied by reusing models pre-trained on similar meteorological datasets.

11. RESOURCES:

The following **web resources** and tools were used throughout this project for algorithm implementation, understanding, and performance evaluation:

Technical Libraries & Tools

- **Python** – Primary programming language
- **scikit-learn (sklearn)** – For Logistic Regression, Naive Bayes, KNN, and Random Forest
Website: <https://scikit-learn.org/>
- **TensorFlow / Keras** – For implementing Artificial Neural Networks
Website: <https://www.tensorflow.org/>
Keras: <https://keras.io/>
- **NumPy** – Numerical operations: <https://numpy.org/>
- **Matplotlib & Seaborn** – For visualizations:
<https://matplotlib.org/>, <https://seaborn.pydata.org/>

Learning & Documentation

- **Scikit-learn Documentation**
<https://scikit-learn.org/stable/documentation.html>
- **Keras API Docs**
<https://keras.io/api/>
- **Medium Articles & Towards Data Science** – For concept clarity and tutorials
<https://towardsdatascience.com>
- **GeeksforGeeks – Machine Learning Section**
<https://www.geeksforgeeks.org/machine-learning>

11. CONCLUSION:

1. This project aimed to build and evaluate a weather condition prediction system using various machine learning algorithms.
2. Several models were implemented: Logistic Regression, Naive Bayes, K-Nearest Neighbors (KNN), Random Forest, and Artificial Neural Networks (ANN).
3. Logistic Regression worked well on simple, linearly separable data and was easy to interpret.
4. Naive Bayes was very fast but didn't perform well when feature independence was violated.
5. KNN gave good results on small datasets but was slow and prone to overfitting when k was too small.
6. Random Forest performed best overall, handling complex data and avoiding overfitting.



7. ANN showed the highest accuracy and F1-scores but needed more training time and careful tuning.
8. All models were tested using standard evaluation metrics like accuracy, precision, recall, F1-score, ROC, and Precision-Recall curves.
9. A user-input prediction interface was created using the best model, showing real-world application of the system.
10. In the future, the system could be improved with more tuning, better feature selection, use of ensemble methods like XGBoost, and deployment through a web interface or API.